

密 级： 内部

阶 段： 概要设计阶段

版 次： V1.0

TH-Scheduler-周期任务调度器
软件概要设计说明
产品型号-Scheduler

共 XX 页

太行项目组

2026-05-06

TH-Scheduler-周期任务调度器

软件概要设计说明

产品型号-Scheduler

编制：日期：

校对：日期：

审核：日期：

标审：日期：

审定：日期：

批准：日期：

目 录

1 范围.....	1
1.1 标识.....	1
1.2 系统概述.....	1
1.3 文档概述.....	1
2 引用文档.....	1
3 系统设计决策.....	2
4 系统体系结构设计.....	2
4.1 调度器 CSCI.....	3
4.1.1 CSCI 概述.....	3
4.1.2 CSCI 部件设计.....	3
4.1.3 执行方案.....	3
4.1.4 接口设计.....	3
5 需求可追踪性.....	4
6 注释.....	5

1 范围

1.1 标识

本文档适用于周期性任务调度器软件（Scheduler），软件名称为周期性任务调度器，缩写为 Scheduler，文档标识号为 TH/Scheduler-SDD-V1.0，版本号为 V1.0。该软件由太行项目组嵌入式软件研发部研制，用于嵌入式 Linux 系统中驱动电池仿真模型以固定周期运行。

- a) 文档标识号：TH/Scheduler-SDD-V1.0；
- b) 标题：软件概要设计说明；
- c) 软件名称：TH-Scheduler-周期任务调度器；
- d) 软件缩写：Scheduler；
- e) 软件版本号：V1.0。

1.2 系统概述

本概要设计说明面向周期性任务调度器软件的总体设计，描述软件在嵌入式 Linux 系统中的定时调度、漂移补偿和电池模型驱动方案。软件基于 clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, ...)实现绝对时间等待，以 25ms 固定周期驱动电池模型运行，具备超时检测和降采样输出能力。

系统运行于 ARM Linux（内核 4.1.15，单核）平台，采用模块化设计将定时器管理、电流序列管理、降采样输出和主循环控制分离，便于后续详细设计和独立测试。软件以 C 语言实现，仅使用 POSIX 标准 API 和 C99 标准。

1.3 文档概述

本文档给出周期性任务调度器软件的概要设计，包括系统设计决策、体系结构设计、CSCI 划分、接口定义、需求可追踪性和相关注释内容。本文档为后续详细设计、编码实现、测试策划和技术评审提供依据。

文档密级为内部，仅用于太行项目组、嵌入式软件开发人员和授权评审人员查阅。

2 引用文档

本章列出概要设计说明编制所直接引用的标准、需求文档及相关支撑资料。

表 2-1 引用文件

序号	名称	版本	发布日期	来源
1	《GJB 438C 软件文档编制规	438C	2024-01-01	相关标准库

	范》			
2	《周期性任务调度器软件需求规格说明》	V1.0	2026-05-06	太行项目组

3 系统设计决策

系统设计采用绝对时间补偿与模块化分离相结合的总体方案：定时器模块基于 `CLOCK_MONOTONIC` 计算绝对唤醒时间点，主循环模块按固定顺序调度电池模型和输出模块。

针对嵌入式实时性和精度要求，系统采用以下关键设计决策：

1. `clock_nanosleep` 绝对时间等待：使用 `TIMER_ABSTIME` 模式，每周期计算 $\text{next_wake} = \text{start} + n * \text{interval}$ ，消除累积漂移。

2. 超时检测与跳过：当任务执行超过当前周期时，跳过延迟等待，打印警告，立即计算下一个绝对唤醒时间。

3. 松耦合集成：调度器通过 `battery_model.h` 声明的函数接口调用电池模型，不直接访问其内部结构体，便于独立开发和测试。

4. 降采样输出策略：内部以 25ms 周期运行，每 40 个周期（1 秒）输出一次数据，减少 I/O 开销。

5. 静态内存分配：不使用动态内存，电流序列通过静态数组定义，定时器状态通过静态结构体管理。

a)关于系统将接收的输入和将产生的输出的设计决策，包括与其他系统、HWCI、CSCI 和用户的接口。如果这一信息的全部或部分已在接口设计说明(IDD)中给出，则可以直接引用。

b)有关响应每个输入或条件的系统行为的设计决策，包括系统要执行的动作、响应时间和其他性能特性，模型化的物理系统的说明，选定的方程式/算法/规则，以及对不允许的输入或条件进行的处理。

c)有关数据库/数据文件如何呈现给用户的设计决策。如果这一信息的全部或部分在数据库设计说明(DBDD)中给出，则可直接引用。

d)为满足安全性和保密性需求所选择的方法。

e)为满足需求所做的其他系统设计决策，例如为提供所需的灵活性、可用性和可维护性所选择的方法。

4 系统体系结构设计

调度器软件采用单 CSCI 体系结构，内部划分为定时器管理部件、电流序

列管理部件和主循环控制部件三个核心部件，通过函数调用接口协同工作。

4.1 调度器 CSCI

4.1.1 CSCI 概述

调度器 CSCI 是系统的唯一软件配置项，负责完成高精度定时调度、电流序列管理、电池模型驱动和降采样输出等全部功能。该 CSCI 封装了基于 `clock_nanosleep` 的绝对时间补偿机制，确保 25ms 周期的长期精确性。

4.1.2 CSCI 部件设计

4.1.2.1 调度控制与定时管理部件

调度控制与定时管理部件由定时器管理子部件、电流序列管理子部件、降采样输出子部件和主循环控制子部件组成。定时器管理子部件封装 `clock_gettime` 和 `clock_nanosleep` 调用，提供 `timer_init()` 和 `timer_wait_next()` 接口；电流序列管理子部件维护时间-电流对照表，提供 `get_current_at_time()` 查询接口；降采样输出子部件按 40 周期间隔输出数据；主循环控制子部件协调所有子部件的执行顺序。

4.1.3 执行方案

调度器 CSCI 按 '初始化→循环调度' 的方式执行。初始化阶段完成电池模型初始化 (`battery_init`)、定时器初始化 (`timer_init(25)`) 和电流序列加载。运行阶段主循环每 25ms 执行一次：调用 `timer_wait_next()` 等待下一周期→获取当前仿真时间和电流值→调用 `battery_step(I_out, DT)`→读取电池状态→累计周期计数并判断是否输出→重复。

4.1.4 接口设计

该 CSCI 对外提供命令行调用入口 (`main` 函数)，通过标准输出打印运行结果。对内与电池模型 CSCI 通过函数接口交互 (`battery_init`、`battery_step`、`getter` 函数)，不直接访问电池模型内部数据。

4.1.4.1 接口标识和接口图

接口标识以 'IF-SCHED-*' 规则统一命名，包括定时器初始化接口 IF-SCHED-TIMER-INIT、定时器等待接口 IF-SCHED-TIMER-WAIT、电流查询接口 IF-SCHED-CURRENT-GET、主循环接口 IF-SCHED-MAIN。接口图以调用链方式描述调度器与电池模型的依赖关系。

4.1.4.1.1 调度器接口

定时器接口 IF-SCHED-TIMER-INIT：函数原型 `void timer_init(int interval_ms)`，输入为周期 (ms)，设定内部状态和起始时间。

定时器接口 IF-SCHED-TIMER-WAIT：函数原型 `void`

timer_wait_next(void), 无输入参数, 阻塞到下一个绝对唤醒时间点。若已超时则打印警告并跳过。

电 流 查 询 接 口 IF-SCHED-CURRENT-GET : 函 数 原 型 double get_current_at_time(double t), 输入为仿真时间 (s), 返回当前电流值 (A)。

电 池 模 型 调 用 接 口 : 通 过 battery_model.h 声 明 的 battery_init() 、 battery_step(double, double) 、 battery_get_voltage() 、 battery_get_soc() 、 battery_get_ploss() 函数与电池模型交互。

5 需求可追踪性

本章建立调度器软件需求与概要设计单元之间的双向追踪关系, 支持从需求到设计、从设计回溯需求的核查分析。

- a)从本 SDD 所标识的每个软件单元, 到分配给它的 CSCI 需求的可追踪性。
- b)从每个 CSCI 需求, 到被分配这些需求的软件单元的可追踪性。

表 5-1 需求的正向追踪性

软件需求		软件单元	
名称/标识	软件需求规格说明的章节号	名称/标识	本文档的章节号
高精度定时器功能	3. 2. 1	定时器管理子部件	4. 1
		定时器管理子部件	4. 1
		电流序列管理子部件	4. 1
		主循环控制子部件	4. 1
降采样输出功能	3. 2. 5	降采样输出子部件	4. 1

表 5-2 需求的逆向追踪性

软件单元		软件需求	
名称/标识	名称/标识	名称/标识	本文档的章节号
定时器管理子部件	4. 1	高精度定时器功能	3. 2. 1
定时器管理子部件	4. 1	时间漂移补偿功能	3. 2. 2
电流序列管理子部件	4. 1	电流序列管理功能	3. 2. 3
主循环控制子部件	4. 1	主循环调度功能	3. 2. 4
降采样输出子部件	4. 1	降采样输出功能	3. 2. 5

6 注释

术语说明：CSCI 指计算机软件配置项；CLOCK_MONOTONIC 指 POSIX 单调递增时钟；TIMER_ABSTIME 指 `clock_nanosleep` 的绝对时间等待模式。

绝对时间补偿算法： $\text{start_time} = \text{clock_gettime}(\text{MONOTONIC})$ ，每次等待目标为 $\text{next_wake} = \text{start_time} + n * \text{interval_ms} / 1000$ ， n 为周期计数。若当前时间已超过 next_wake ，则跳过等待并打印警告。

降采样策略：每 40 个周期（ $40 * 25\text{ms} = 1000\text{ms} = 1\text{s}$ ）输出一次数据，输出格式为'时间(s) SOC 电压(V) 功率(W)'。

本文档中的章节编号、接口标识和软件单元名称均用于概要设计阶段的统一描述，后续详细设计可在保持可追踪性的前提下细化。